National University of Singapore
School of Computing
CS3244: Machine Learning
Solution to Tutorial 10

Reinforcement Learning

1. **Markov Decision Process.** Describe a task that fits into the MDP framework, then identify what are its states, actions, and rewards.

   One could fit the task of playing Mario into the MDP framework. The states can be the frames of the game. The actions are up, down, left, right, jump, and shoot. The reward is the raw score of the game.

2. **Bellman Equations.** Answer the following questions.

   (a) Express v_* in terms of q_*.

   $v_*(s) = \max_a q_*(s, a)$

   (b) Express π_* in terms of q_*.

   $\pi_*(s) = \arg\max_a q_*(s, a)$

   (c) Express q_* recursively in terms of γ and $p(s', r \mid s, a)$.

   $q_*(s, a) = \sum_{s',r} p(s', r \mid s, a)(r + \gamma \max_{a'} q_*(s', a'))$

3. **Reinforcement Learning.** Recall that in reinforcement learning, we do not have access to the transition function $p(s', r \mid s, a)$ and reward function $R(s, a)$. As such, dynamic programming methods are no longer feasible.

   In the following questions, try to devise a reinforcement learning algorithm using the key ideas provided. You may write in psudeocode.

   (a) Instead of solving for $Q(s, a)$, we can estimate it by running through a few episodes. Using the estimate $\hat{Q}(s, a)$, we can greedily update our policy.

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:
  $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
  $Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
  $Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):
  Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
  Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
      Append G to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
      $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

(b) One shortcoming of the previous method is that we can only obtain the estimated $\hat{Q}(s,a)$ after every episode. This will not work on tasks that never end! To overcome this, we have to try to formulate an estimate of $\hat{Q}(s,a)$ after every step taken in our environment. Intuitively, it make sense that our estimate of the expected return $\hat{Q}(s_t, a_t)$ would be better if we take into account the reward we obtained after taking one more step. In other words, the estimator $r_{t+1} + \gamma\hat{Q}(s_{t+1}, a_{t+1})$ is a better estimate of the true $Q(s_t, a_t)$ than $\hat{Q}(s_t, a_t)$.

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s,a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma Q(S', A') - Q(S,A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal